

Exercise 13: Sorting Space Complexity

Setup:

For this exercise, you will run different sorting algorithms and observe how much extra memory they consume. You have been provided with starter code in Canvas.

- Create a new IntelliJ Project for Exercise 13. Create a package with your username and move the downloaded starter files into your package.

Instructions:

SpaceComplexityDriver.java

Read the comments at the start of SpaceComplexityDriver before running the program. Run SpaceComplexityDriver.java. The program will pause and tell you to hit "Enter" to proceed. Because the program is still currently running, we can attach a JConsole session to it, to monitor the resources it is using.

Launch JConsole (see instructions below) and click on the process called SpaceComplexityDriver.java. Click on the "insecure connection" button. If you did not run the file before this step, you will not see the program in the list of running processes. If you need to stop the SpaceComplexityDriver program and re-run it, you can reattach the JConsole without rerunning the executable by clicking on the New Connection option in the upper left.

Once JConsole is running you will see a several windows monitoring different resources. We will be focusing on the memory window, but it could be useful to everyone in a while glance at the CPU usage. You can also choose to click on the memory tab so it is the only window displayed.

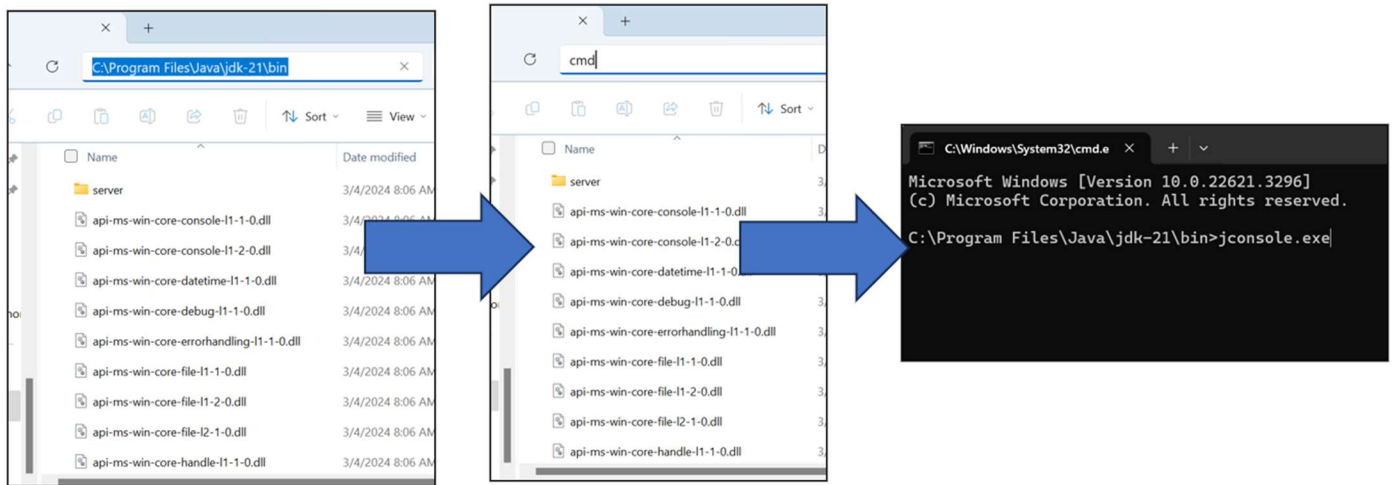
Wait until the memory usage levels out and is stable; it should only take a second. Hit "Enter" to run the first sort. Wait a second or two for the memory usage to level out again, it probably won't drop, and then hit "Enter" again to run the next sorting algorithm. Do this for all the sorting algorithms and make a mental note of any spikes that occur right after running a sort. Take a screenshot of the memory window and add it to your Word document submission. On the screenshot annotate (arrows and text) any sharp rises that were caused by a sort and which sort caused them.

Run SpaceComplexityDriver.java for list sizes (numElements) of 1_000, 10_000, 100_000, and 1_000_000. For 1_000_000 only run it for shell and merge sort. Comment the code for selection and insertion.

JConsole

JConsole is a tool that allows you to profile the resources, such as memory and the cpu, that a java program uses. To attach JConsole to a program (process), it first must be running. See running SpaceComplexityDriver.java above. The JConsole executable is located in the bin/ folder of your Java installation. If Java was installed correctly and the bin/ folder was added to your PATH, you should be able to open a command terminal and type JConsole to launch the application.

If the bin/ folder is not in your PATH, you will need to navigate to this folder and launch the executable from there. Open up a Windows folder and navigate to the location of your Java installation. Go into the bin/ folder. Clear the address bar at the top and type 'cmd' to launch a command terminal at this location. You should be able to type JConsole in the command prompt and it should run the JConsole executable. See screenshots below.



Analysis

1. Based on our discussion of the sort algorithms in lecture and the provided code, which sorting algorithms would require additional space beyond the list itself. Explain your answer.
2. Do your screenshots confirm your analysis?

Submission

Add the answers for your analysis along with the annotated screenshots to a Word document. Turn the Word document in to a pdf and submit the pdf.